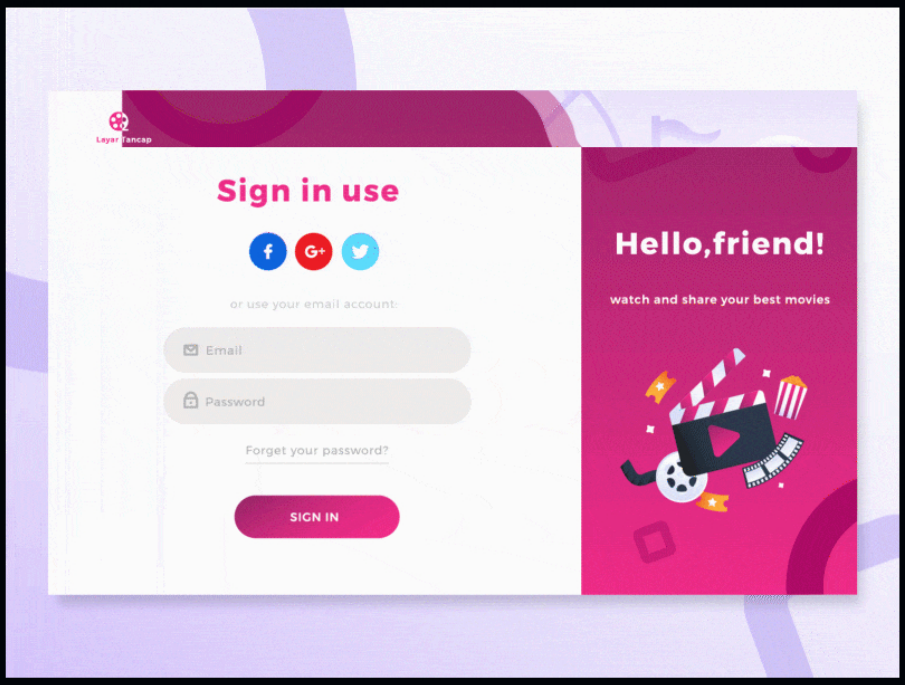


1. Introduction to CSS

What is CSS?

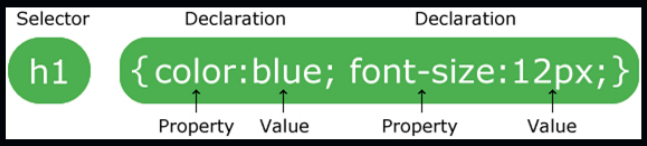
- CSS stands for `Cascading Style Sheets`.
- It is a styling language used to control the layout and appearance of web pages written in `HTML` or `XML`.
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS saves a lot of work. It can control the layout of multiple web pages all at once

CSS Syntax

```
/* SYNTAX */
selector {
  property: value;
}
```

```
- A CSS rule-set consists of a selector and a declaration block:

- The selector points to the HTML element you want to style.

- The declaration block contains one or more declarations separated by semicolons.

- Each declaration includes a CSS property name and a value, separated by a colon.

- Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.
```

Example



Common Properties

- `color` : Sets the color of the text.
- `background-color` : Sets the background color of an element.
- `font-size` : Sets the size of the font.
- `height` : Sets the height of an element.
- `width` : Sets the width of an element.

Examples

```
p {
  color: blue;
  background-color: red;
  font-size: 24px;
  height: 100px;
  width: 200px;
}
```

`Ways` to use CSS

1. Inline Styles

- Apply CSS styles directly to an HTML element using the `style` attribute.
- Example:

```
<!-- index.html -->
<p style="color: blue; font-size: 24px;">This is a paragraph.</p>
```

2. Internal Stylesheet

- Write CSS code directly in the `<head>` section of your HTML file using the `<style>` tag.
- Example:

```
<!-- index.html -->
<head>
  <style>
    body {
      background-color: #f2f2f2;
    }
  </style>
</head>
```

3. External Stylesheet

- Create a separate CSS file and link it to your HTML file using the `<link>` tag.
- Example:

```
/* styles.css */
body {
  background-color: #f2f2f2;
}
```

```
<!-- index.html -->
<head>
  <link rel="stylesheet" type="text/css" href="styles.css">
</head>
```

Types of `Comments` in CSS

```
* In programming languages, a comment is a piece of code that is not executed by the compiler or interpreter.

* Comments are used to explain the code, make it more readable, and provide information to other developers.
```

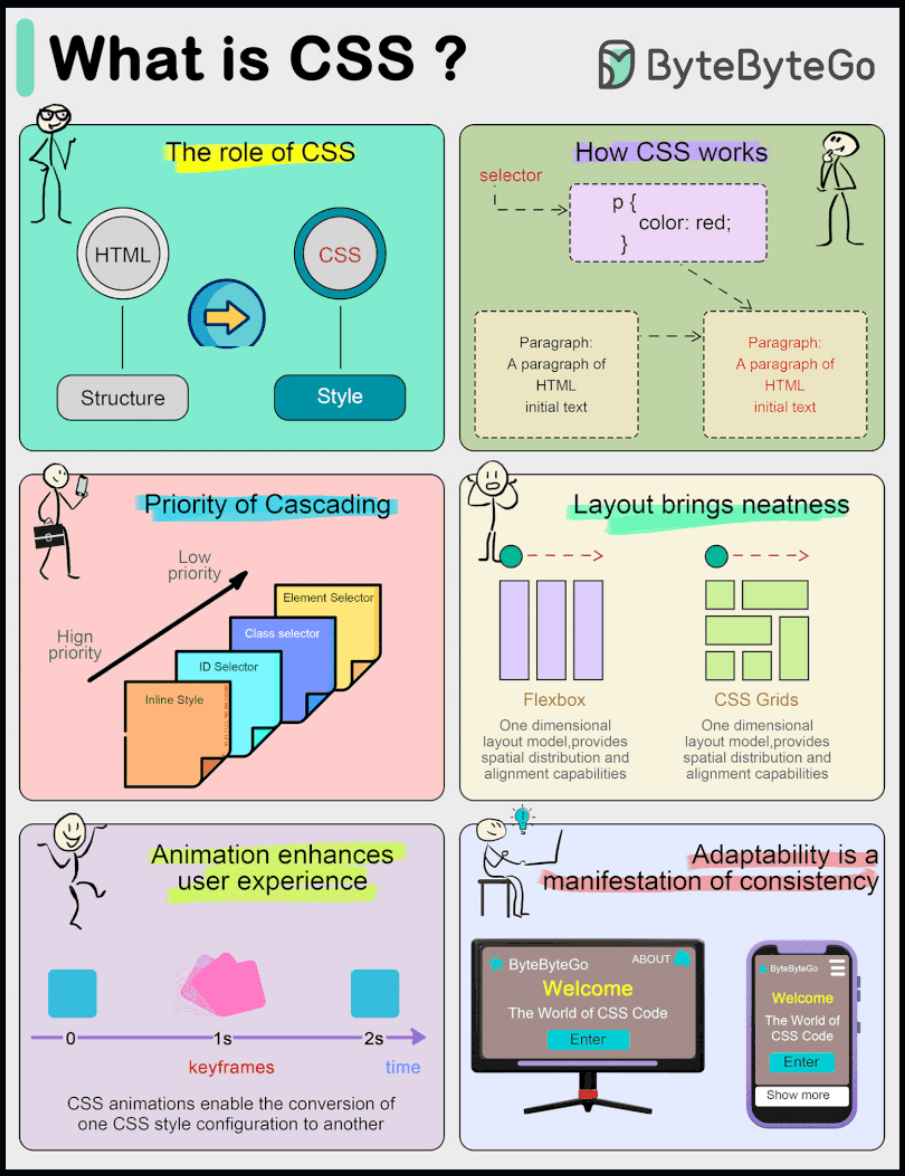
Only one way to comment (Single-Line & Multi-Line) Comments

- Use `/* */` to comment out lines of code.

> Example:

```
/*
This is a multi-line comment
that spans multiple lines
*/
body {
  background-color: #f2f2f2;
}
```

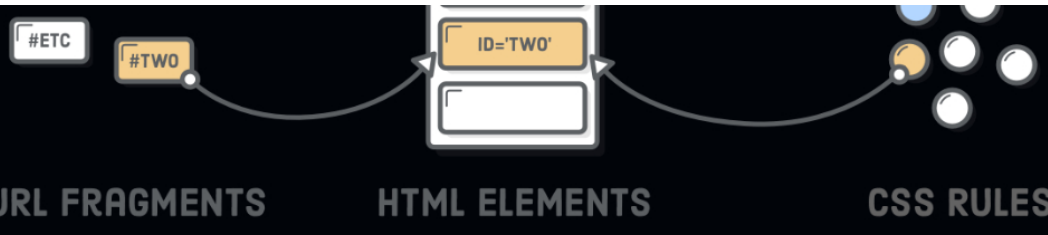
Working of CSS



2. Selectors

Definition

```
Selectors are patterns used to select the elements you want to style in your HTML document.

They can target elements based on their type, class, id, attributes, and more.
```



`Types of Selectors`

Selector Type	Description	Example
`Element` Selector	Selects elements by their tag name.	`p { color: blue; }`
`Class` Selector	Selects elements by their class attribute.	`.class-name { ... }` or `div.class-name { ... }`
`ID` Selector	Selects elements by their id attribute.	`#id-name { ... }`
Attribute Selector	Selects elements by the presence or value of an attribute.	`[attribute-name] { ... }`
`Universal` Selector	Selects all elements.	`* { ... }`
Grouping Selector	Applies styles to multiple selectors at once.	`h1, h2, h3 { ... }`
Descendant Combinator	Selects elements that are descendants of another element.	`div p { ... }`

3. Box Model

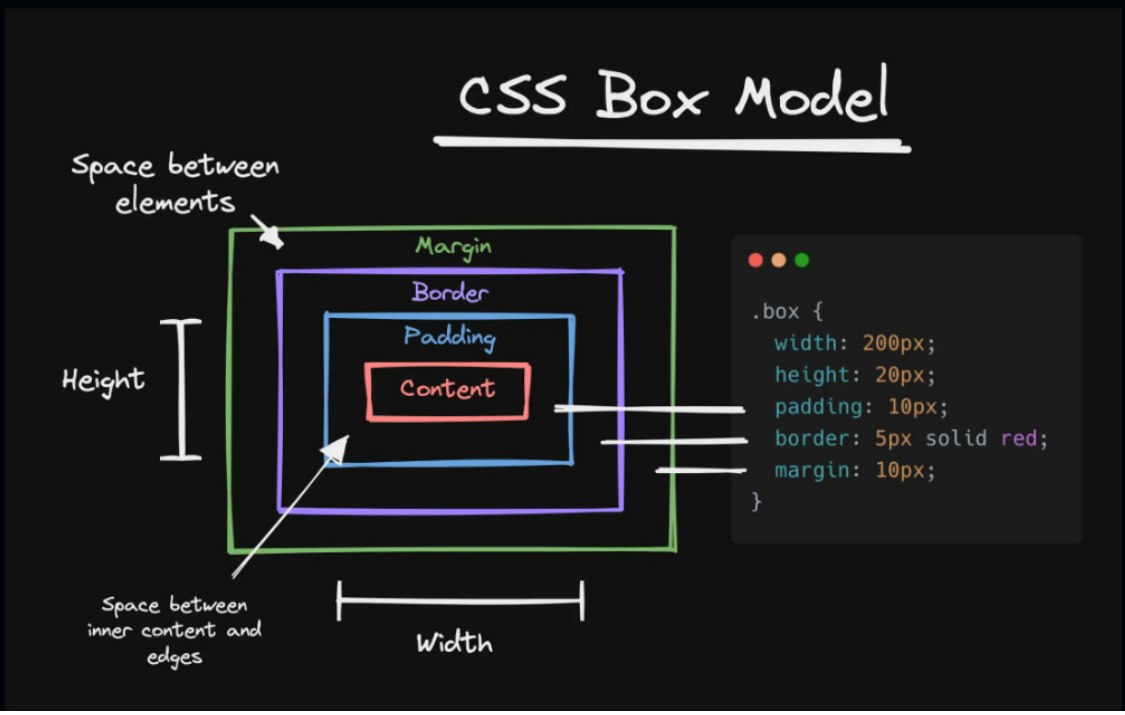
What is the Box Model?

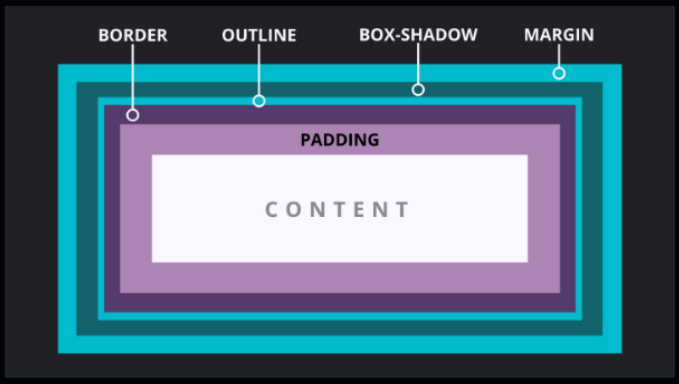
The box model is a concept in CSS that describes the structure of an HTML element as a rectangular box.

Components of the Box Model

- **Content Area**: The area where the content of the element is displayed.
- **Padding**: The space between the content area and the border.
- **Border**: The visible outline of the element.
- **Margin**: The space between the element and other elements.
- **Outline**: A line that is drawn around elements, outside the border edge.
- **Box Sizing**: Determines how the width and height of an element are calculated.
- **Box Shadow**: Adds a shadow effect to the box.
- **Height and Width**: Sets the height and width of the element.

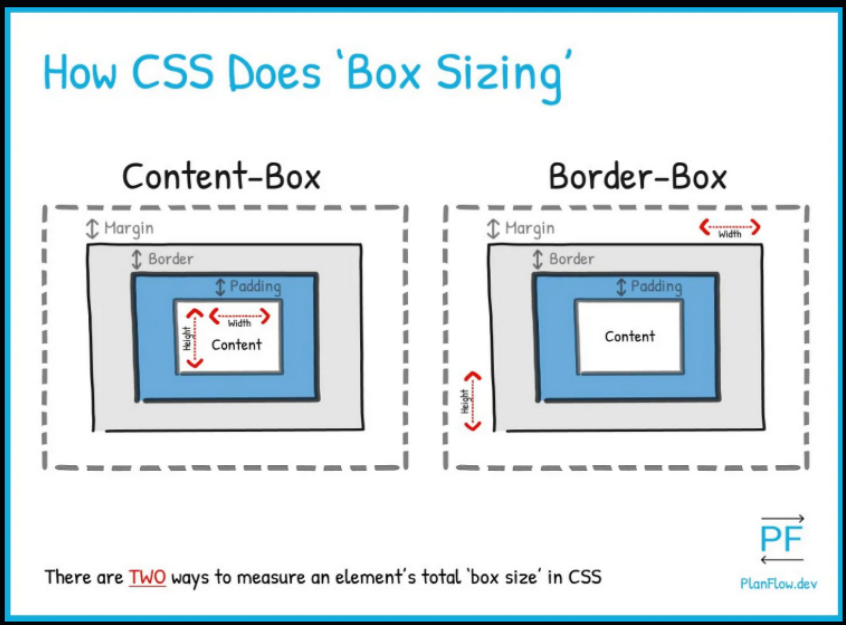
> Image of `box model`





Examples

```
div {
  width: 200px;
  height: 100px;
  padding: 20px;
  border: 1px solid black;
  margin: 10px;
}
```



4. Properties and Values

`color` Properties

- `color` : Sets the color of the text. (e.g., `color: blue;`)
- `background-color` : Sets the background color of an element. (e.g., `background-color: red;`)
- `border-color` : Sets the color of the border. (e.g., `border-color: green;`)
- `outline-color` : Sets the color of the outline. (e.g., `outline-color: yellow;`)
- `text-shadow-color` : Sets the color of the text shadow. (e.g., `text-shadow: 2px 2px 4px #000000;`)
- `box-shadow-color` : Sets the color of the box shadow. (e.g., `box-shadow: 10px 10px 5px #888888;`)

>>> types of value in [color]

color generator | Site

- **Keyword Values**: Use predefined color keywords, such as `red` , `blue` , `green` , etc.
- **Hexadecimal Values**: Use hexadecimal codes, such as `#FF0000` , `#008000` , `#0000FF` , etc.
- **RGB Values**: Use RGB (Red, Green, Blue) values, such as `rgb(255, 0, 0)` .
- **RGBA Values**: Use RGBA (Red, Green, Blue, Alpha) values, such as `rgba(255, 0, 0, 0.5)` .
- **HSL Values**: Use HSL (Hue, Saturation, Lightness) values, such as `hsl(240, 100%, 50%)` .
- **HSLA Values**: Use HSLA (Hue, Saturation, Lightness, Alpha) values, such as `hsla(0, 100%, 50%, 0.5)` .

`Background` Properties

- `background-color` : Sets the background color of an element [`all values as color properties`].
- `background-image` : Sets the background image of an element [`url(<image-path>)`].
- `background-repeat` : Sets how the background image should be repeated [`repeat` , `no-repeat` , `repeat-x` , `repeat-y`].
- `background-position` : Sets the position of the background image [`left` , `right` , `top` , `bottom` , `center`].
- `background-attachment` : Sets whether the background image should be fixed or scroll with the content [`fixed` , `scroll`].
- `background-size` : Sets the size of the background image [`cover` , `contain` , `auto`].
- `background-origin` : Sets the origin of the background image [`border-box` , `padding-box` , `content-box`].
- `background-clip` : Sets the clip of the background image [`border-box` , `padding-box` , `content-box` , `text`].
- `background` : Sets all background properties in one declaration [`color image repeat position attachment size origin clip`].

Example Visit

```css
body {
  /* Set background color */
  background-color: #f2f2f2;

  /* Set background image */
  background-image: url('image.jpg');

  /* Set background repeat */
  background-repeat: no-repeat;

  /* Set background position */
  background-position: center;

  /* Set background attachment */
  background-attachment: fixed;

  /* Set background size */
  background-size: cover;

  /* Set background origin */
  background-origin: border-box;

  /* Set background clip */
  background-clip: padding-box;
  /* or */
  /* Set background clip for text */
  background-clip: text;
  color: transparent;
  text-shadow:none;

  /* Shorthand background property */
  background: #f2f2f2 url('image.jpg') no-repeat center fixed;
}
```

`Height` and `Width` Properties

- `height` : Sets the height of an element.
- `width` : Sets the width of an element.
- `min-height` : Sets the minimum height of an element.
- `max-height` : Sets the maximum height of an element.
- `min-width` : Sets the minimum width of an element.
- `max-width` : Sets the maximum width of an element.

`Padding` Properties

- `padding` : Sets the padding of an element.
- `padding-top` : Sets the top padding of an element.
- `padding-right` : Sets the right padding of an element.
- `padding-bottom` : Sets the bottom padding of an element.
- `padding-left` : Sets the left padding of an element.
- `Padding-inline` : Sets the padding on the inline axis of an element.
- `padding-block` : Sets the padding on the block axis of an element.
- `padding: 10px 20px 30px 40px;` is equivalent to [`top right bottom left` | `top-bottom right-left` | `all` | `top right-left bottom`].

`Margin` Properties

- `margin` : Sets the margin of an element.
- `margin-top` : Sets the top margin of an element.
- `margin-right` : Sets the right margin of an element.
- `margin-bottom` : Sets the bottom margin of an element.

- `margin-left` : Sets the left margin of an element.
- `margin: auto` : Sets the margin to auto, which centers the element horizontally.
- `margin-inline` : Sets the margin on the inline axis of an element.
- `margin-block` : Sets the margin on the block axis of an element.
- `margin: 10px 20px 30px 40px;` is equivalent to [`top right bottom left | top-bottom right-left | all | top right-left bottom`].

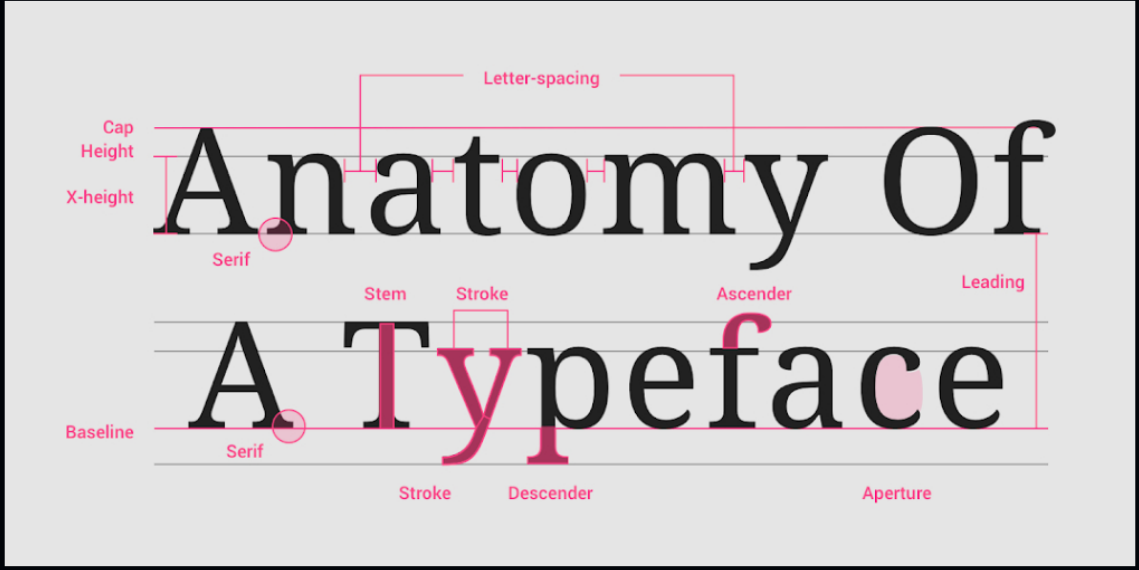
`Border` Properties

- `border` : Sets the border of an element [`width style color`].
- `border-width` : Sets the width of the border.
- `border-style` : Sets the style of the border [`solid` , `dashed` , `dotted` , `double` , `groove` , `ridge` , `inset` , `outset` , `none` , `hidden`].
- `border-color` : Sets the color of the border.
- `border-*` : Sets the border of a specific side [`* -> top|right|bottom|left`].
- `border-*-**` :Sets any side and property of the border [`** -> width|style|color|radius`].
- `border-radius` | `border-*-radius` : Sets the radius of the border
  [`all-side | tl+br tr+bl | tl tr+bl br | (length/percentage)`]
  { `* --> top-left | top-right | bottom-left | bottom-right` }.
  - `length(x-axis)` : Specifies the radius in pixels or other units.
  - `percentage(y-axis)` : Specifies the radius as a percentage of the element's width or height.

Examples

```
div {
  /* Set border */
  border: 1px solid black;

  /* Set border width */
  border-width: 2px;

  /* Set border style */
  border-style: dashed;

  /* Set border color */
  border-color: blue;

  /* Set border radius */
  border-radius: 10px;


/* start */
border-radius: 1em/5em;
/* It is equivalent to:  */
border-top-left-radius: 1em 5em;
border-top-right-radius: 1em 5em;
border-bottom-right-radius: 1em 5em;
border-bottom-left-radius: 1em 5em;
/* end */

  /* Set border of a specific side */
  border-top: 2px solid red;
}
```

`Text` & `Font` Properties

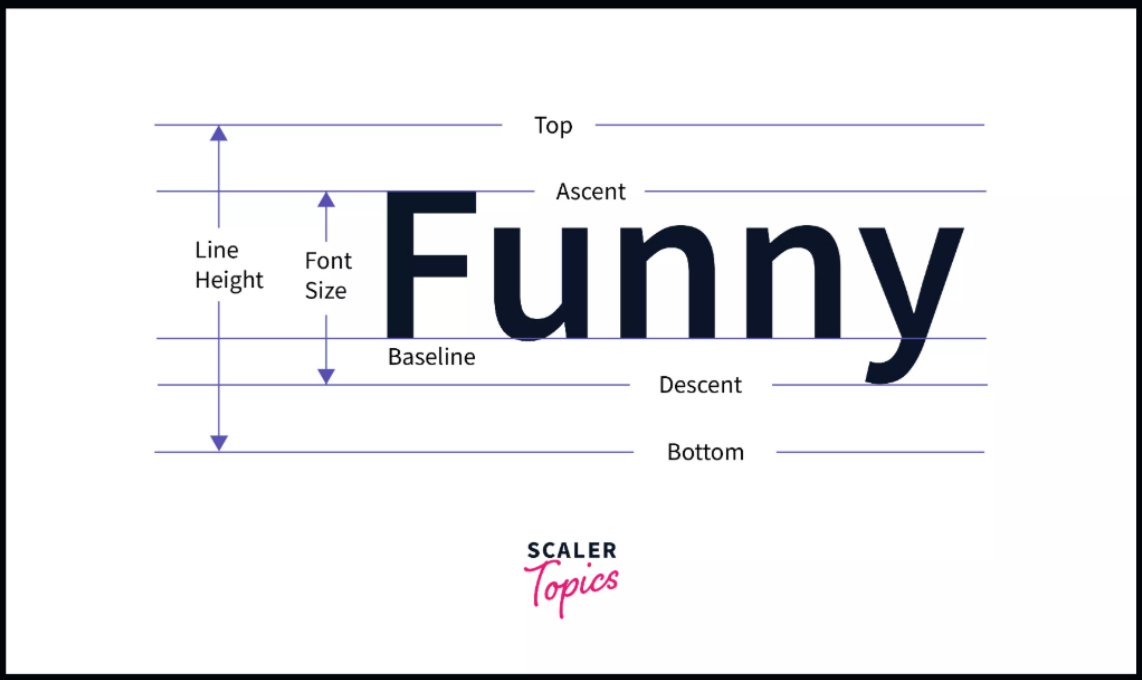


Text Properties

- `color` : Sets the color of the text.
- `text-align` : Aligns the text (`left` , `right` , `center` , `justify`).
- `text-decoration` : Adds decoration (`color|style|line|thickness`).
  - `text-decoration-color` : Sets the color of the text decoration.
  - `text-decoration-style` : Sets the style of the text decoration (`solid` , `dashed` , `dotted` , `double` , `wavy`).
  - `text-decoration-thickness` : Sets the thickness of the text decoration.
  - `text-decoration-line` : Specifies the line(s) to be decorated (`underline` , `overline` , `line-through` , `blink`).
- `text-transform` : Controls capitalization (`uppercase` , `lowercase` , `capitalize`).
- `text-shadow` : Adds shadow to text.
- `letter-spacing` : Sets space between characters(`length`).
- `text-overflow` : Specifies how overflowed content that is not displayed should be signaled to the user (`clip` , `ellipsis`).
- `text-wrap` : Controls how text wraps within an element (`wrap` , `nowrap`).
- `word-spacing` : Sets space between words(`length`).
- `write-mode` : Sets the direction of text (`horizontal-tb` , `vertical-rl` , `vertical-lr`).
- `line-height` : Sets the height of lines of text(`length`).
- `text-indent` : Indents the first line of text(`length`).

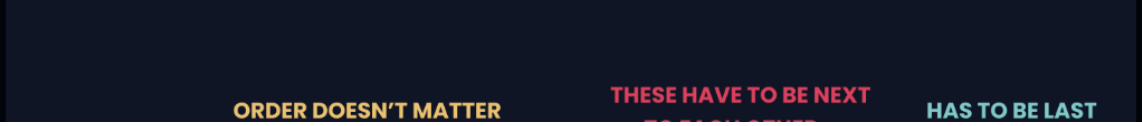
Example

```css
p {
  color: #333;
  text-align: center;
  text-decoration: underline;
  text-transform: uppercase;
  text-shadow: 2px 2px 5px #aaa;
  letter-spacing: 2px;
  word-spacing: 10px;
  line-height: 1.5;
  text-indent: 30px;
}
```



Font Properties

- `font-family` : Specifies the font(`family name`).
- `font-size` : Sets the size of the font.
- `font-style` : Sets the style (`normal` , `italic` , `oblique`).
- `font-weight` : Sets the thickness (`normal` , `bold` , `bolder` , `lighter` , `100` – `900`).
- `font-variant` : Sets small-caps (`normal` , `small-caps`).
- `font-stretch` : Expands or condenses the font(`expanded` , `condensed` , `ultra-condensed`).
- `font` : Shorthand for all font properties(`font-style | font-variant | font-weight | [font-size]/line-height | [font-family]`).



Example

```
font family import from google font
```

```
/* online */
/* custom.css */
@import url('https://fonts.googleapis.com/css2?family=Bitcount+Grid+Double:wght@100..900&family=Edu+NSW+ACT+Cursive:wght@400..7

/* offline */
@import url('/Fira_Code/FiraCode-VariableFont_wght.ttf');

/* or */
@font-face{
  font-family: 'myfont';
  src: url('/Mine/Mine-VariableFont_wght.ttf') format('truetype');
}

h1 {
  font-family: 'Arial','myfont',"Fira Code", sans-serif;
  font-size: 2em;
  font-style: italic;
  font-weight: bold;
  font-variant: small-caps;
  font-stretch: expanded;
}
```

```
<!-- x.html -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Bitcount+Grid+Double:wght@100..900&family=Edu+NSW+ACT+Cursive:wght@400..70

<style>
  h1 {
    font-family: 'Bitcount Grid Double', cursive;
    font-weight: bold;
  }
</style>
```

Shadows Properties

Box Shadow

- Adds shadow to elements.
- Syntax: `box-shadow: h-offset v-offset blur spread color inset;`
- Values Pattern:
  - `h-offset` : Horizontal offset (e.g., `4px`).
  - `v-offset` : Vertical offset (e.g., `4px`).
  - `blur` : Blur radius (e.g., `10px`).
  - `spread` : Spread radius (e.g., `2px`).
  - `color` : Color of the shadow (e.g., `#888888`).
  - `inset` : Optional keyword to make the shadow an inner shadow.

Example

```
div {
  box-shadow: 4px 4px 10px 2px #888888;
}
```

Text Shadow

- Adds shadow to text

- Adds shadow to text.
- Syntax: `text-shadow: h-offset v-offset blur color;`
- Values Pattern:
  - `h-offset` : Horizontal offset (e.g., `2px`).
  - `v-offset` : Vertical offset (e.g., `2px`).
  - `blur` : Blur radius (e.g., `5px`).
  - `color` : Color of the shadow (e.g., `#555`).

Example

```
h2 {
  text-shadow: 2px 2px 5px #555;
}
```

Drop Shadow

- Applies a shadow effect to an element.
- Syntax: `filter: drop-shadow(h-offset v-offset blur color);`

Example

```
img {
  filter: drop-shadow(2px 2px 5px #000);
}
```

`Overflow` Properties

- Controls what happens when content overflows an element's box.
- `overflow` : `visible` , `hidden` , `scroll` , `auto`
- `overflow-x` : Controls horizontal overflow.
- `overflow-y` : Controls vertical overflow.

Example

```
div {
  width: 200px;
  height: 100px;
  overflow: auto;
}
```

`Positioning` Porperties

Types of Positioning

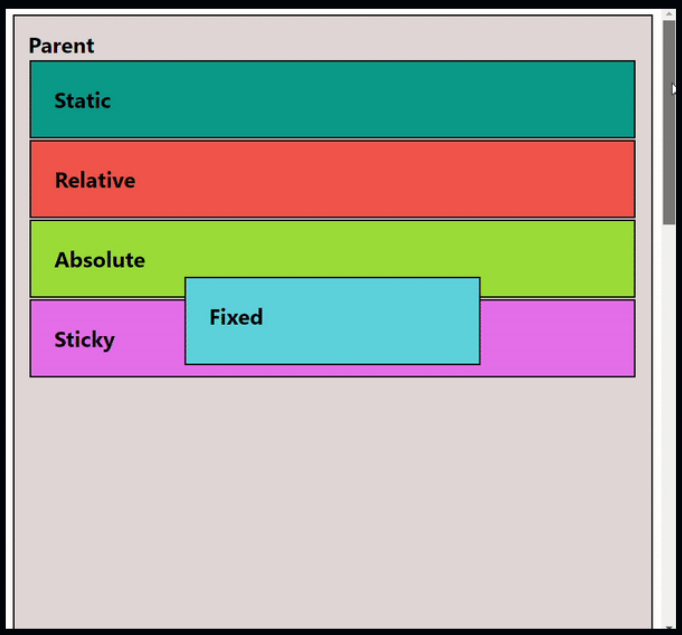
- **Static Positioning**: The default positioning scheme.

- **Relative Positioning**: Positions an element relative to its normal position.

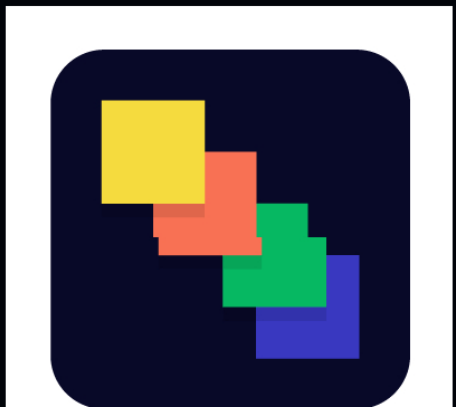
- **Absolute Positioning**: Positions an element relative to its nearest positioned ancestor.

- **Fixed Positioning**: Positions an element relative to the viewport.

- **Sticky Positioning**: A hybrid of relative and fixed positioning.



- **Z-index**: Controls the stacking order of positioned elements.



Examples

```
/* Relative Positioning */
.relative {
  position: relative;
  top: 20px;
  left: 30px;
}

/* Absolute Positioning */
.absolute {
  position: absolute;
  top: 20px;
  left: 30px;
}

/* Fixed Positioning */
.fixed {
  position: fixed;
  top: 20px;
  left: 30px;
}
```

`Display` Properties

- `display` : Sets the display property of an element.
- `visibility` : Sets the visibility of an element.
- `opacity` : Sets the opacity of an element.

Display Property Values

- `block` : Displays an element as a block element.
- `inline` : Displays an element as an inline element.
- `inline-block` : Displays an element as an inline-block element.
- `flex` : Displays an element as a flexible container.
- `grid` : Displays an element as a grid container.
- `table` : Displays an element as a table.
- `table-cell` : Displays an element as a table cell.
- `table-column` : Displays an element as a table column.
- `table-row` : Displays an element as a table row.
- `none` : Hides an element.

Visibility Property Values

- `visible` : Makes an element visible.
- `hidden` : Hides an element.
- `collapse` : Collapses an element.

Opacity Property Values

- `0.0` : Makes an element completely transparent.
- `1.0` : Makes an element completely opaque.

Examples

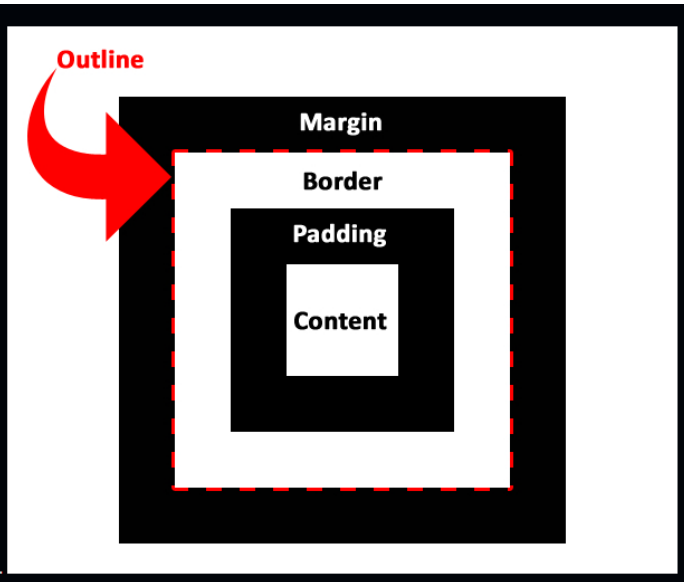
```
div {
  /* Set display property */
  display: block;

  /* Set visibility property */
  visibility: visible;

  /* Set opacity property */
  opacity: 0.5;
}
```

Outline Properties

An outline is a line that is drawn around elements, OUTSIDE the borders, to make the element "stand out".

- `outline` : Sets the outline of an element.
- `outline-width` : Sets the width of the outline.
- `outline-style` : Sets the style of the outline.

- `outline-color` : Sets the color of the outline.



- `outline-offset` : Sets the offset of the outline.

Examples

```
body{
    /* Set outline */
  outline: 1px solid black;

  /* Set outline width */
  outline-width: 2px;

  /* Set outline style */
  outline-style: dashed;

  /* Set outline color */
  outline-color: blue;

  /* Set outline offset */
  outline-offset: 5px;
}
```

Gradiants Properties

- A gradient is a gradual transition between two or more colors.
- Gradients are commonly used for backgrounds, buttons, and other design elements to create a visually appealing effect.

Types of Gradients

- **Linear Gradient**: A gradient that transitions colors along a straight line.
  - `Syntax` : linear-gradient(direction, color-stop1, color-stop2, ...)
  - Example:

```
background: linear-gradient(to right, red, blue);```
```

- **Radial Gradient**: A gradient that transitions colors in a circular or elliptical shape.
  - `Syntax` : radial-gradient(shape size at position, start-color, ..., last-color)
  - Example:

```
background: radial-gradient(circle, red, blue);```
```

- **Conic Gradient**: A gradient that transitions colors around a central point, creating a cone-like effect.
  - `Syntax` : conic-gradient(from angle at position, color-stop1, color-stop2, ...)
  - Example:

```
background: conic-gradient(from 0deg at center, red, yellow, green, blue, red);
```

Filters Properties

- Filters are used to apply visual effects to elements, such as blurring, brightness, contrast, and more.

Syntax

```
/* Apply filter effects means applying filter effects to the element */
filter: function(value) function(value) ...;
/* Apply backdrop filter effects means applying filter effects to the background */
backdrop-filter: function(value) function(value) ...;
```

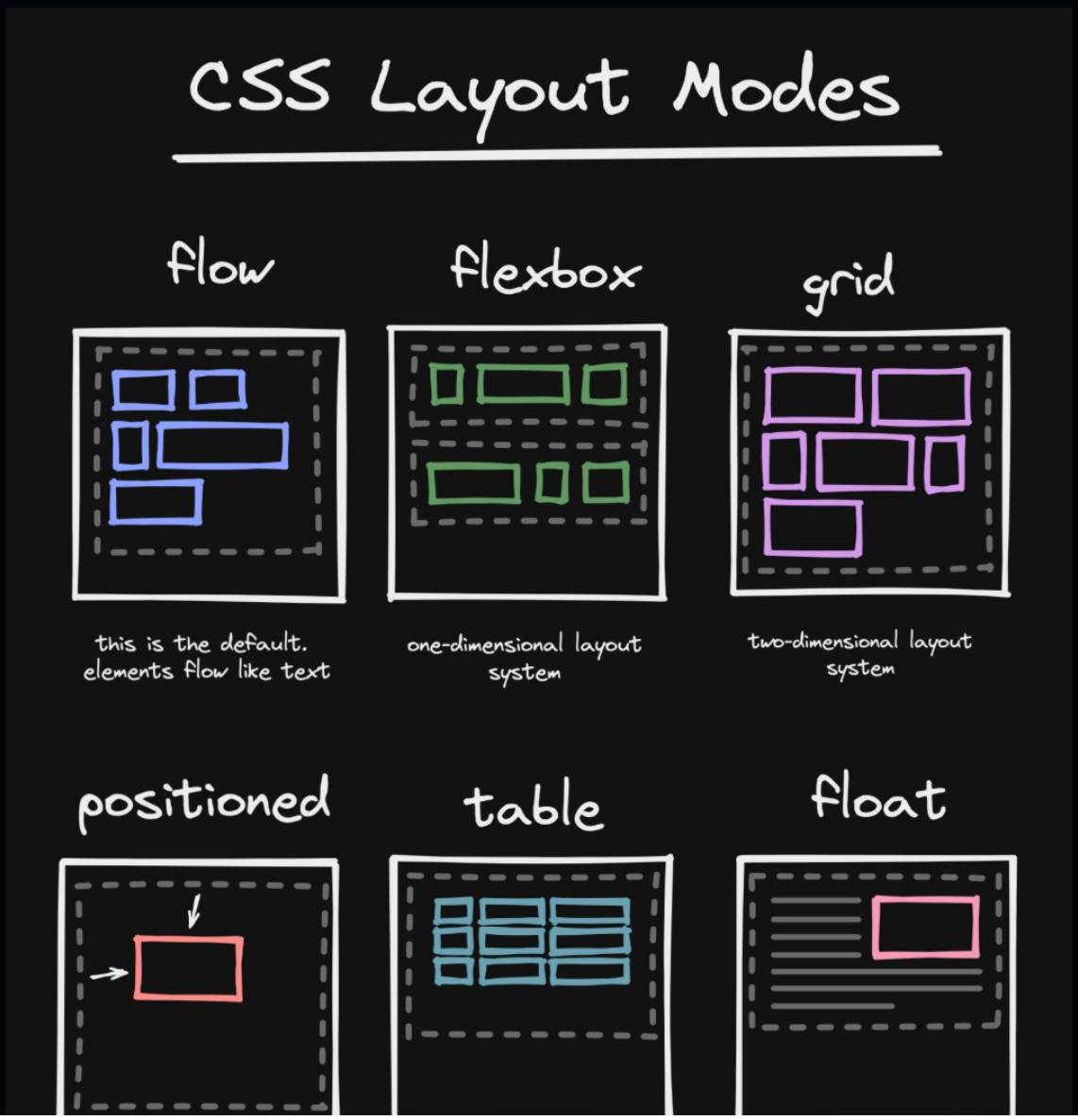
Common Filter Functions

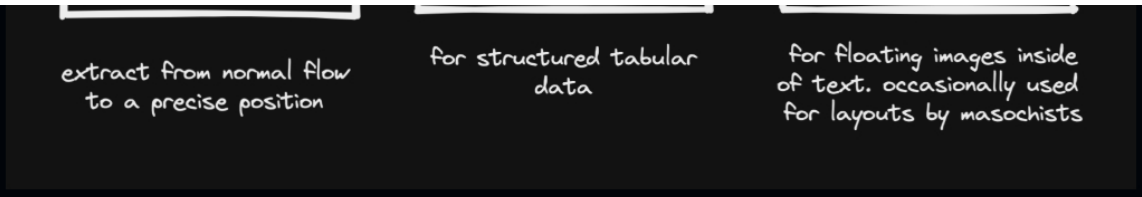
- `blur(px)` : Applies a Gaussian blur to the element.
- `brightness(%)` : Adjusts the brightness of the element.
- `contrast(%)` : Adjusts the contrast of the element.
- `grayscale(%)` : Converts the element to grayscale.
- `invert(%)` : Inverts the colors of the element.
- `sepia(%)` : Applies a sepia tone to the element.
- `saturate(%)` : Adjusts the saturation of the element.
- `hue-rotate(deg)` : Rotates the hue of the element.
- `drop-shadow(offset-x offset-y blur-radius color)` : Applies a drop shadow to the element.
- `opacity(%)` : Adjusts the opacity of the element.

Example

```css
/* single filter function */
div{
filter:url('filter.svg#filter-id');
}

/* // Apply multiple filters to an image */
img {
  filter: blur(5px) brightness(150%) contrast(200%) grayscale(50%) sepia(30%) saturate(200%) hue-rotate(90deg) drop-shadow(5px
}
```

5. Layout





`Float` Properties

What is Float?

Float is a CSS property that allows an element to be placed to the left or right of its parent container, and allows other elements to wrap around it.

Properties

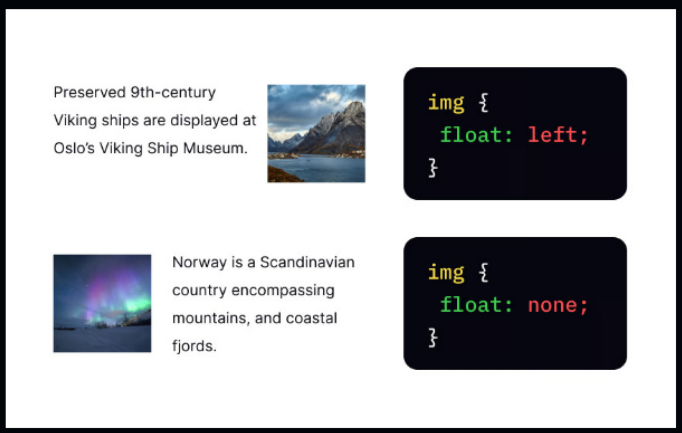
- `float` : Sets the float property of an element.
- `clear` : Sets the clear property of an element, which specifies whether an element should be moved below a floating element.

Float Property Values

- `left` : Floats an element to the left of its parent container.
- `right` : Floats an element to the right of its parent container.
- `none` : Does not float an element.
- `inherit` : Inherits the float property from the parent element.

Clear Property Values

- `left` : Moves an element below a floating element on the left.
- `right` : Moves an element below a floating element on the right.
- `both` : Moves an element below all floating elements.
- `none` : Does not move an element below a floating element.



Examples

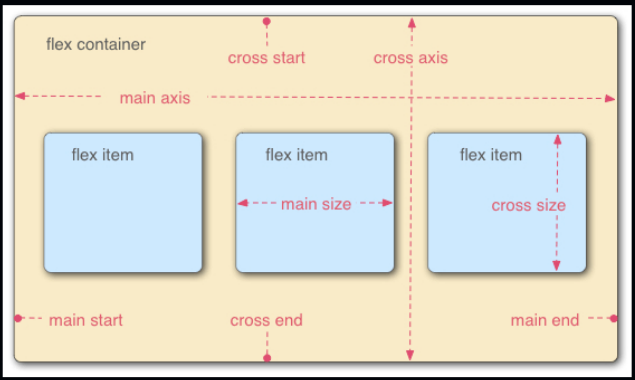
```
.float-left {
  float: left;
}

.float-right {
  float: right;
}

.clear-left {
  clear: left;
}

.clear-right {
  clear: right;
}

.clear-both {
  clear: both;
}
```

`Flexbox` Properties

What is Flexbox?

Flexbox is a powerful layout module that makes it easy to design flexible and responsive layouts.

Flexbox Model



Flex Container (Parent) Properties

Property	Description	Example Value
`display`	Defines a flex container.	`flex`, `inline-flex`
`flex-direction`	Sets the direction of flex items.	`row`, `row-reverse`, `column`, `column-reverse`
`flex-wrap`	Controls whether flex items wrap onto multiple lines.	`nowrap`, `wrap`, `wrap-reverse`
`justify-content`	Aligns items horizontally (main axis).	`flex-start`, `flex-end`, `center`, `space-between`, `space-around`, `space-evenly`
`align-items`	Aligns items vertically (cross axis).	`flex-start`, `flex-end`, `center`, `baseline`, `stretch`
`align-content`	Aligns lines of items when there is extra space in the cross axis (multi-line).	`flex-start`, `flex-end`, `center`, `space-between`, `space-around`, `stretch`
`gap`	Sets spacing between flex items.	`10px`, `1em`

Flex Item (Child) Properties

Property	Description	Example Value
`flex-grow`	Defines how much a flex item will grow relative to the rest.	`0`, `1`, `2`
`flex-shrink`	Defines how much a flex item will shrink relative to the rest.	`0`, `1`, `2`
`flex-basis`	Sets the initial main size of a flex item.	`auto`, `100px`
`flex`	Shorthand for `flex-grow`, `flex-shrink`, and `flex-basis`.	`1 0 100px`
`align-self`	Overrides `align-items` for individual flex items.	`auto`, `flex-start`, `flex-end`, `center`, `baseline`, `stretch`
`order`	Controls the order in which flex items appear.(-ve value allowed)	`0`, `1`, `2`

Demonstration

CSS Tricks Flexbox Guide

Flex box Demonstration

Flexbox Froggy game for learn flex

Example: Parent and Child Flexbox Properties

```
.container {
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: 20px;
}

.item {
  flex: 1 1 150px;      /* flex-grow: 1; flex-shrink: 1; flex-basis: 150px; */
  align-self: flex-start;
  order: 2;
}
```

```
.item.special {
  flex: 2 0 200px;
  align-self: center;
  order: 1;
}
```

7. `Transform` Properties

What are Transform?

- Transform allows you to modify the coordinate space of the CSS visual formatting model.

Properties

- `transform` : Applies a 2D or 3D transformation to an element.

Common Transform Functions

- `translate(x, y)` : Moves an element from its current position.
- `rotate(angle)` : Rotates an element around a fixed point.
- `scale(x, y)` : Scales an element in the X and Y directions.
- `skew(x-angle, y-angle)` : Skews an element along the X and Y axes.

Example

```
div {
  transform: translate(50px, 100px) rotate(45deg) scale(1.5, 1.5);
}
```

6. `Transitions` Properties

What are Transitions?

- Transitions allow property changes in CSS values to occur smoothly over a specified duration.

Properties

- `transition-property` : The property to transition.
- `transition-duration` : How long the transition takes.
- `transition-timing-function` : The speed curve (`ease` , `linear` , `ease-in` , `ease-out` , `ease-in-out`).
- `transition-delay` : Delay before the transition starts.
- `transition` : Shorthand for all transition properties.

Example

```
button {
  background-color: blue;
  transition: background-color 0.3s ease;
}
button:hover {
  background-color: green;
}
```

---

7. `Animations` Properties

What are Animations?

- Animations allow you to change CSS properties over time using keyframes.

Properties

- `@keyframes` : Defines the animation.
- `animation-name` : Name of the keyframes.
- `animation-duration` : How long the animation lasts.
- `animation-timing-function` : Speed curve.
- `animation-delay` : Delay before animation starts.
- `animation-iteration-count` : Number of times animation runs.

- `animation-direction` : Direction (`normal` , `reverse` , `alternate` , `alternate-reverse`).
- `animation-fill-mode` : How styles are applied before/after animation.
- `animation-play-state` : Pauses or plays the animation (`running` , `paused`).
- `animation-timeline` : A list of animations to apply to the element.
- `animation` : Shorthand for all animation properties (`name` , `duration` , `timing-function` , `delay` , `iteration-count` , `direction` , `fill-mode` , `play-state` , `timeline`).

Example

```
<div id="container">
  <div id="square"></div>
  <div id="stretcher"></div>
</div>
```

```
#container {
  height: 300px;
  overflow-y: scroll;
  scroll-timeline-name: --square-timeline;
  position: relative;
}

/* timeline and play-state example */
#square {
  background-color: deeppink;
  width: 100px;
  height: 100px;
  margin-top: 100px;
  animation-name: rotateAnimation;
  animation-duration: 1ms; /* Firefox requires this to apply the animation */
  animation-direction: alternate;
  animation-timeline: --square-timeline;

  position: absolute;
  bottom: 0;
}

@keyframes rotateAnimation {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

#stretcher {
  height: 600px;
}
```

```
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
div {
  animation-name: fadeIn;
  animation-duration: 2s;
  animation-iteration-count: 1;
}
```

8.Media Queries Properties

What are Media Queries?

Media queries are a way to apply different styles based on different conditions, such as screen size or device type.

`Syntax`

```
@media (condition) {
  /* styles */
}
```

Examples

```
@media (max-width: 768px) {
  /* styles for small screens */
}

@media (min-width: 1024px) {
```

```
  /* styles for large screens */
}
```